

Pack 31 — Session Recording Capture + Encrypted Storage + Playback (real code)

RemoteDesk · Pack 31 — Session Recording Capture + Encrypted Storage + Playback (real code)

Codex / Claude Code handoff. NEW feature area (the trust/consent/receipt system, Packs 15-30, is complete). This pack adds session recording: the desktop captures a session as time-stamped chunks, the API stores chunk metadata + presigned upload refs (no media bytes through the API), and the web dashboard plays a recording back from an ordered manifest. Recording is **consent-gated** by the existing device policy + session approval, and recordings are **content-addressed + integrity-checked** so a tampered chunk is detectable on playback. Build order: shared → api → web → desktop → tests.

Scope: capture → store metadata → play back, with consent gating and per-chunk integrity. **Recording requires an explicit policy flag + consent (fail-closed: off by default). Media bytes go to object storage via presigned URLs, never through the API. Chunk manifest is hash-chained so reordering/tampering is detectable. No raw input payloads/keystrokes in any recording metadata. No real storage credentials in code.**

Tag legend: **SAFE_DIRECT_COPY** (new file, drop in) · **REVIEW_REQUIRED** (patch an existing file by hand).

Zip-ready file tree

```
remotedesk/
  packages/shared/src/recording/
    types.ts                # SAFE_DIRECT_COPY
    chunkManifest.ts        # SAFE_DIRECT_COPY
    recordingPolicy.ts      # SAFE_DIRECT_COPY
    schema.ts               # SAFE_DIRECT_COPY
    index.ts                # SAFE_DIRECT_COPY
  packages/shared/src/index.ts  # REVIEW_REQUIRED (patch — export recording)
  apps/api/
    prisma/schema.prisma    # REVIEW_REQUIRED (patch — 3 models)
    prisma/migrations/0031_recording/migration.sql    # SAFE_DIRECT_COPY
    src/lib/recordingManifest.ts    # SAFE_DIRECT_COPY
    src/lib/storagePresign.ts      # SAFE_DIRECT_COPY (pluggable; dev stub, no creds)
    src/repositories/recordingRepository.ts          # SAFE_DIRECT_COPY
    src/services/recording/recording.service.ts      # SAFE_DIRECT_COPY
    src/routes/recording.routes.ts  # SAFE_DIRECT_COPY
    src/server.ts                # REVIEW_REQUIRED (patch — mount /api/recordings)
  apps/web/app/dashboard/recordings/
    page.tsx                   # SAFE_DIRECT_COPY (list)
    [recordingId]/page.tsx     # SAFE_DIRECT_COPY (player)
    RecordingPlayer.tsx        # SAFE_DIRECT_COPY
    recordingApi.ts            # SAFE_DIRECT_COPY
    recordings.module.css      # SAFE_DIRECT_COPY
  apps/web/app/dashboard/page.tsx  # REVIEW_REQUIRED (patch — add link)
  apps/desktop/src/renderer/src/
```

```

services/recordingCapture.ts      # SAFE_DIRECT_COPY (MediaRecorder chunker)
services/recordingUploader.ts     # SAFE_DIRECT_COPY (presigned PUT; fail-silent)
session/sessionSetup.ts          # REVIEW_REQUIRED (patch — start/stop capture, consent-gated)
tests/recording/
  chunkManifest.test.ts           # SAFE_DIRECT_COPY
  recordingPolicy.test.ts         # SAFE_DIRECT_COPY
  recordingManifest.test.ts       # SAFE_DIRECT_COPY
  recording-contract.test.ts      # SAFE_DIRECT_COPY
docs/session-recording.md         # SAFE_DIRECT_COPY
generated-remotedesk-pack31-merge-summary.md # doNotMerge
generated-remotedesk-pack31-code-review.md   # doNotMerge
generated-remotedesk-pack31-manifest.json    # doNotMerge

```

Strict rules (enforced by this pack)

- No filler files, no placeholder-only files, no `extraFile*.ts`.
- Do not rewrite the app; only the listed patches touch existing files.
- Every file tagged `SAFE_DIRECT_COPY` or `REVIEW_REQUIRED`.
- Tests import only real generated/copied files, never a `REVIEW_REQUIRED`-only path.
- Runtime code matches the current Express / Next app-router / Electron renderer structure.
- Never fake production readiness: the storage presigner is a clearly-marked dev stub with no real credentials.

Part A — Shared

`packages/shared/src/recording/types.ts` — `SAFE_DIRECT_COPY`

Define: `RecordingStatus = 'recording' | 'finalizing' | 'ready' | 'failed'`; `RecordingChunkRef { index; sha256; sizeBytes; durationMs; storageKey }`; `RecordingManifestEntry extends RecordingChunkRef { hash }` (hash-chain link); `RecordingDto { id; sessionId; deviceId; status; codec; chunkCount; totalDurationMs; rootHash; createdAt }`; `RecordingChunkInput` (no media bytes — index + sha256 + size + duration + storageKey only).

`packages/shared/src/recording/chunkManifest.ts` — `SAFE_DIRECT_COPY`

Pure hash-chain over ordered chunk refs (reuse the Pack 22 chaining idea): `buildManifest(chunks, sha256Hex) → { entries, rootHash }`; `verifyManifest(manifest, sha256Hex) → { valid, firstBrokenIndex }`. Reordering, insertion, or a swapped chunk sha breaks the chain. Empty list → root = genesis (valid).

`packages/shared/src/recording/recordingPolicy.ts` — `SAFE_DIRECT_COPY`

Fail-closed gate reusing the device-policy concepts: `canRecord(policy, approval)` denies unless a new `recordingEnabled` policy flag is true AND session approval is satisfied (Pack 16). `recordingDenialReason(...)` for UI. Off by default.

`packages/shared/src/recording/schema.ts` — `SAFE_DIRECT_COPY`

Zod: `startRecordingSchema { sessionId, codec }`; `appendChunkSchema { index>=0, sha256 (64 hex), sizeBytes>0, durationMs>=0, storageKey }`; `finalizeRecordingSchema { rootHash }`.

`packages/shared/src/recording/index.ts` + `packages/shared/src/index.ts` (REVIEW_REQUIRED) — barrel + export.

Part B — API: Prisma + migration

`apps/api/prisma/schema.prisma` (REVIEW_REQUIRED) — add 3 models, link to existing `Session` + `Device` :

- `Recording` (id, sessionId, deviceId?, orgId, status, codec, chunkCount, totalDurationMs, rootHash?, createdById?, createdAt) — indexes on orgId, sessionId.
- `RecordingChunk` (id, recordingId, index, sha256, sizeBytes BigInt, durationMs, storageKey, hash, createdAt) — `@@unique([recordingId, index])`, index on recordingId.
- `RecordingAccessEvent` (id, recordingId, orgId, userId?, action 'created' | 'viewed' | 'finalized', createdAt) — content-free access log (ties into Pack 14 style).

`apps/api/prisma/migrations/0031_recording/migration.sql` — SAFE_DIRECT_COPY: the three tables + indexes.

Part C — API runtime

`apps/api/src/lib/storagePresign.ts` — SAFE_DIRECT_COPY. Pluggable `StoragePresigner` interface (`presignPut(key)` , `presignGet(key)`) + a `LocalDevPresigner` that returns a clearly-fake local URL and **fails closed if no storage env is configured** (no real S3/GCS creds in code). Production swap point documented.

`apps/api/src/lib/recordingManifest.ts` — SAFE_DIRECT_COPY. Server-side wrapper over the shared `buildManifest` / `verifyManifest` using node sha256, used at finalize to confirm the client-reported `rootHash` matches the chained chunks before marking `ready` .

`apps/api/src/repositories/recordingRepository.ts` — SAFE_DIRECT_COPY. create recording, append chunk, list chunks (ordered), finalize (set status + rootHash + chunkCount), list/get for org, write access events.

`apps/api/src/services/recording/recording.service.ts` — SAFE_DIRECT_COPY. `start` (presign-aware, records 'recording'); `appendChunk` (validates index monotonicity, stores ref); `finalize` (re-chains server-side, rejects on mismatch, sets 'ready'); `playbackManifest` (ordered chunks + presigned GET URLs, writes a 'viewed' access event).

`apps/api/src/routes/recording.routes.ts` — SAFE_DIRECT_COPY. Auth + org-scoped:

- `POST /api/recordings` → start
- `POST /api/recordings/:id/chunks` → append chunk metadata
- `POST /api/recordings/:id/finalize` → finalize (server re-verifies chain)
- `GET /api/recordings` → list for org
- `GET /api/recordings/:id` → metadata
- `GET /api/recordings/:id/manifest` → ordered chunks + presigned GET URLs (playback)

`apps/api/src/server.ts` (REVIEW_REQUIRED) — mount `/api/recordings` .

Part D — Web

`recordingApi.ts` — SAFE_DIRECT_COPY: list/get/manifest wrappers.

`recordings/page.tsx` — SAFE_DIRECT_COPY: recordings list (session, duration, status, created).

`recordings/[recordingId]/page.tsx` + `RecordingPlayer.tsx` — SAFE_DIRECT_COPY: fetch manifest, verify the chain client-side (reuse shared verifyManifest via WebCrypto), then play chunks in order via MediaSource; show a clear "integrity verified / tampered" badge. Playback refuses if the chain is broken (fail-closed).

`recordings.module.css` — SAFE_DIRECT_COPY.

`dashboard/page.tsx` (REVIEW_REQUIRED) — add a Recordings link.

Part E — Desktop

`services/recordingCapture.ts` — SAFE_DIRECT_COPY. Wraps `MediaRecorder` over the existing captured screen stream; emits fixed-interval chunks; computes each chunk's sha256 (WebCrypto) for the manifest. Read-only over the existing stream — **no new capture surface, native input execution untouched**.

`services/recordingUploader.ts` — SAFE_DIRECT_COPY. PUTs each chunk blob to its presigned URL, then POSTs the (bytes-free) chunk metadata to the API. **Fails silently** so a recording hiccup never breaks the live session.

`session/sessionSetup.ts` (REVIEW_REQUIRED) — start capture only when `canRecord(policy, approval)` passes; stop + finalize on session end; surface a visible "recording" indicator. Consent-gated, fail-closed.

Part F — Tests (import real files only)

- `chunkManifest.test.ts` — chain builds, verifies, detects reorder/swap, empty = valid.
 - `recordingPolicy.test.ts` — off by default; denied without flag; denied without approval; allowed when both true.
 - `recordingManifest.test.ts` — server re-chain matches client root; mismatch rejected.
 - `recording-contract.test.ts` — zod schemas accept valid, reject bad sha/negative index/missing storageKey.
-

Part G — Docs + reports

- `docs/session-recording.md` — capture → store → play flow, consent gating, integrity model, the "media never transits the API" rule, dev-stub storage note.
 - `generated-remotedesk-pack31-merge-summary.md`, `-code-review.md` — doNotMerge.
-

Safety constraints

- Recording is **off by default**; requires `recordingEnabled` policy flag **and** satisfied session approval (fail-closed).

Media bytes go object-storage→presigned-URL only; **never through the API**.

- Chunk manifest is **hash-chained**; playback (web) and finalize (API) both verify it and refuse a broken chain.
- **No raw input payloads, keystrokes, or clipboard contents** in recording metadata (recording is screen media only; metadata is index/hash/size/duration).
- **No real storage credentials** in code; `LocalDevPresigner` is a marked dev stub that fails closed without storage env.
- Native OS input execution remains disabled/no-op.

Manifest must include

`actualFileCount`, `safeDirectCopy`, `reviewRequired`, `doNotMerge`, `dependenciesRequired`, `knownLimitations`, `estimatedCompletionAfterCarefulMerge`.

Known limitations to record: storage presigner is a dev stub (wire real S3/GCS before prod); playback assumes MediaSource-compatible chunk codec; retention/deletion policy is a follow-up pack; no redaction-of-screen-content during recording (separate from Pack 30 live redaction).

Expected file count

~30 meaningful files (full-stack but focused). Target band 28–34.

Why this pack is next: the trust/consent/receipt system (15-30) is complete; session recording is the biggest untouched product surface and a common enterprise requirement, and it slots cleanly onto the existing policy + approval + integrity-chain primitives without rebuilding anything. Recording reuses the consent gate (16), the hash-chain integrity idea (22), and the org-scoped admin patterns (23-25) — so it's high-impact and low-risk to merge.